

Game AI Programming
Assignment 2 - Pathfinding

Catja Alicia Larsson

larc4t-4@student.ltu.se

December 2025

Table of Contents

1. Problem Specification
2. User Guide
3. System Description
4. Algorithm Description
 - 4.1 A* Algorithm & Dijkstra's Algorithm
 - 4.2 Breadth First Search & Depth First Search
5. Discussion
6. Appendices

1. Problem specification

For this assignment the goal was to explore the performance of four different Pathfinding algorithms on three maps of different sizes and layouts. For performance both calculation time and number of nodes is checked.

2. User Guide

- The solution file for the project is found in the root folder of the GitHub repository, and is called HelloWorld.sln.
- What map and algorithm is run is decided in the MainGame.cpp file. The selection is defined by one variable each, and comments above the variable informs the user of what values can be input, and what those values correspond to.
- There is an extra file called themaps.h, it is not currently relevant to the solution, but contains a possible improvement on how to set the maps, (provided by Fredrik Lindahl).
- There is also a comment at the bottom of the file MainGame.cpp which also contains alternative code for setting up the maps, (also provided by Fredrik Lindahl).

3. System Description

- The project code is split into five relevant files;
 - MainGame.cpp, which runs the main loop of the game. It also contains the arrays that hold the nodes and makes up the different maps, and is responsible for setting up and drawing the maps.
 - node.h and node.cpp, which holds the code for the node class, which defines each square of the map, and holds information on its position and its index in the array, if it is blocked, has been visited, variables to hold how far it is from the start and end nodes, as well as function for getting and setting these variables.
 - algorithm.h and algorithm.cpp contains the class with the four algorithms, A*, Breadth First Search (BFS), Depth First Search (DFS), and Dijkstra's Algorithm. Each Algorithm is defined with its own function, but they share two functions for checking if a node should be added to the open list, and another for adding said node depending on its current status. The class also holds two vectors for keeping track of the nodes to visit, and those that have already been visited.

4. Algorithm Description

For this assignment three search algorithms were pre-decided, the A*, BFS, and DFS. The fourth algorithm chosen is Dijkstras, which is a simplified version of A*.

4.1 A* Algorithm & Dijkstra's Algorithm

When implementing the A* algorithm there is more than one way of deciding when to end the search. For this implementation the search ends once the goal is the smallest node in the list of opened nodes. This might not necessarily result in the shortest path to the goal, but is most often good enough. It can be terminated both later and earlier, with costs to either performance or result, so the middle ground was chosen.

The difference between A* and Dijkstra's in this implementation is that A* looks at a total value of the distance traveled from the start node, and the manhattan distance to the goal node, while Dijkstra's only look at the distance traveled from the start node, when choosing the next node to look at from the list of open nodes.

4.2 Breadth First Search(BFS) & Depth First Search(DFS)

In this test both BFS and DFS were implemented using an iterative approach. DFS could also be implemented recursively. The main difference between the two algorithms in their iterative version is that BFS uses a queue, first in first out method for choosing the next node to look at, while DFS uses a stack, last in first out.

This had one important effect when choosing the order in which neighboring nodes are added to the list. For BFS, to get the best resulting path, the adjacent nodes should be added first, and then the diagonal. However if this order was followed in DFS the resulting path zigg-zagged whenever possible, therefore the diagonal nodes needed to be added first.

5. Discussion

Looking at the time performance of the four different algorithms on the three maps we can see that in map 1 A* had a major lead compared to the other algorithms, with BFS and DFS having similar performance, and Dijkstra's performance only slightly better than them.

In map 2 A* still performs the best, but the advantage it has is much smaller than it was for map 1, and Dijkstra's perform almost as good. BFS and DFS still have a similar performance to each other, and are only slightly slower than A* and Dijkstra's.

However, in map 3 A* actually performed the worst, while Dijkstra's took first place in time, and BFS and DFS coming second.

The likely reason for A* lacking performance in map 3 is probably that map 3 has multiple concave corners in the middle of the map, which can take several iterations before a dead-end is discovered, taking unnecessary time. While BFS and DFS has to look at all nodes before the path is found, they do not have to perform any comparisons when deciding what node to look at next, which cuts down on their time. The interesting part is that Dijkstra's perform the same checks as A*, but still outperforms BFS and DFS.

A quick look at the maps shows that both map 1 and map 3 are mostly open, but map 1 only has convex shapes, which does not result in dead ends, while map 3 has a lot of walls which can result in dead ends for the algorithms. Map 2 differs more from the other two with very limited walkable paths and no large open areas.

The strong limitations of map 2 is likely the reason why the different algorithms get much more similar results when getting the path.

Even though BFS and DFS perform similar in time, the resulting paths are not comparable in this use case. DFS will not return the shortest path in all cases. It gives a somewhat reasonable path in map 2, but in both map 1 and map 3 it will take unnecessary loops, which make the resulting path unreasonably long.

While A* is the fastest in both map 1, the path it returns is slightly longer than the one suggested by BFS and Dijkstra's.

In contrast, in map 3, while being slightly slower in BFS, the path it gives is slightly slower.

6. Appendices

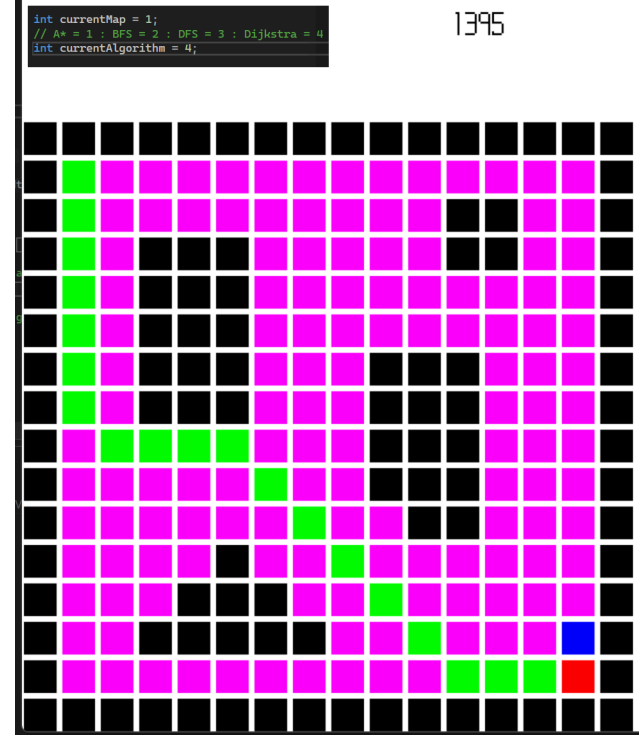
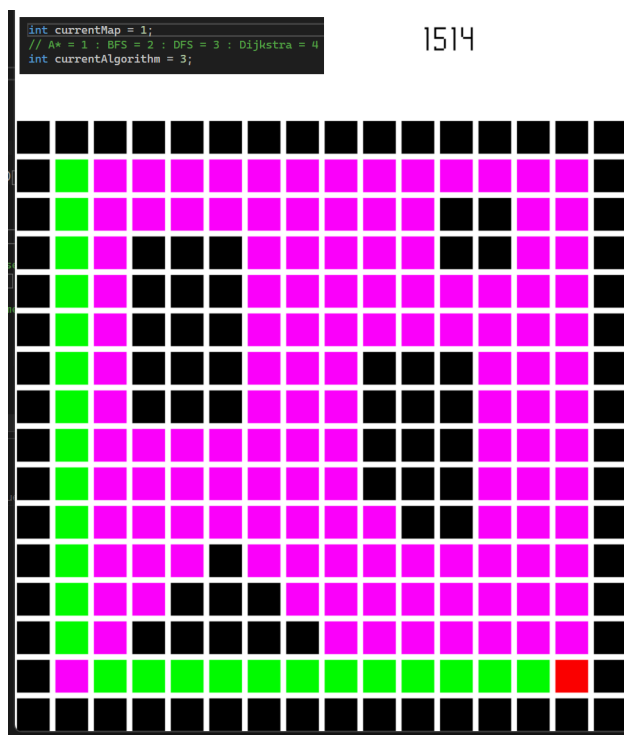
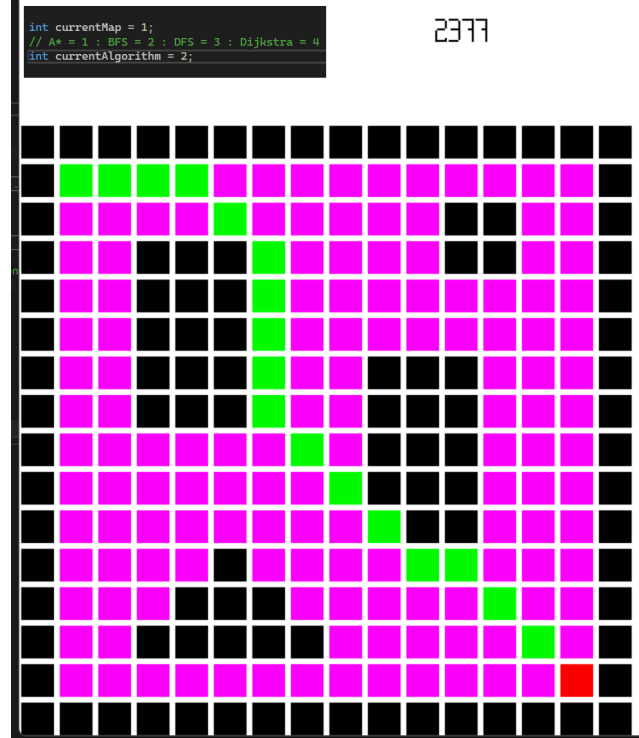
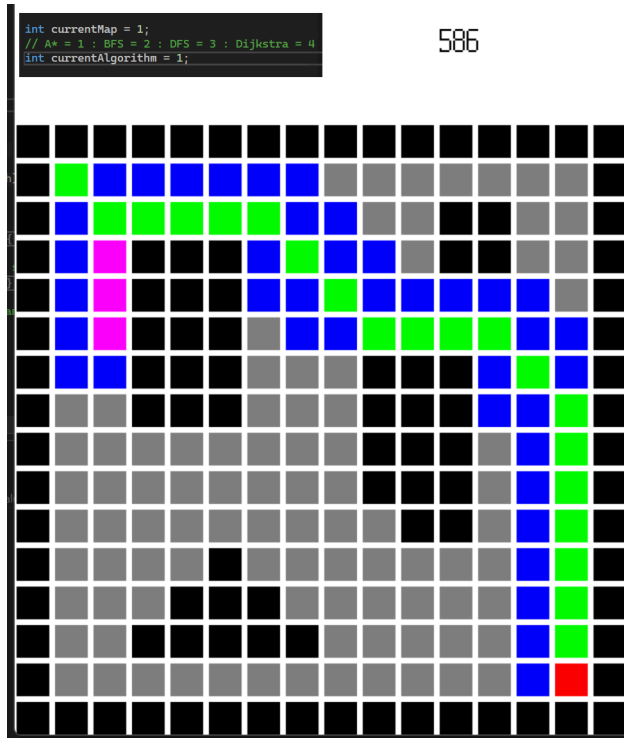
	Run 1	Run 2	Run 3	Run 4	Run 5	Run 6	Run 7	Run 8	Run 9	Run 10	Average	Path length
M1 - A1	611	567	567	571	580	559	565	577	575	573	574.5	22.4
M1 - A2	1474	1478	1465	1462	1440	1464	1507	1451	1446	1428	1461.5	21.8
M1 - A3	1511	1443	1486	1450	1476	1544	1555	1473	1475	1465	1487.8	50.2
M1 - A4	1355	1433	1381	1378	1381	1377	1457	1369	1370	1385	1388.6	21.8
M2 - A1	1055	976	977	983	987	986	972	978	975	980	986.9	43.4
M2 - A2	1265	1083	1066	1074	1065	1073	1109	1116	1045	1061	1095.7	43.4
M2 - A3	1089	1088	1082	1079	1084	1073	1095	1085	1076	1101	1085.2	55.4
M2 - A4	1015	1008	1006	999	1007	999	1056	981	1058	1011	1014	43.4
M3 - A1	4339	4367	4357	4329	4342	4292	4391	4236	4288	4318	4325.9	43.6
M3 - A2	4263	4269	4292	4298	4255	4287	4252	4361	4337	4250	4286.4	44.4
M3 - A3	4340	4296	4301	4281	4338	4264	4268	4218	4241	4271	4281.8	77
M3 - A4	4026	3955	3953	3951	3999	4118	3906	3962	3966	3915	3975.1	43.6

The above table shows the result of ten test runs for each combination of map and algorithm, as well as the calculated average and the length of the resulting path.

M1 stands for map 1, M2 for map 2, and M3 for map 3.

A1 is the A* algorithm, A2 BFS, A3 DFS, and A4 Dijkstra's.

Map 1 results



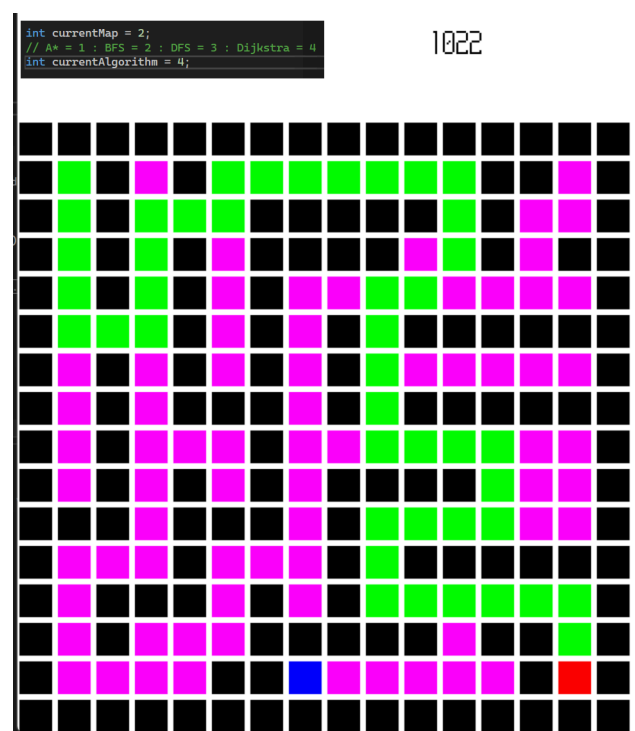
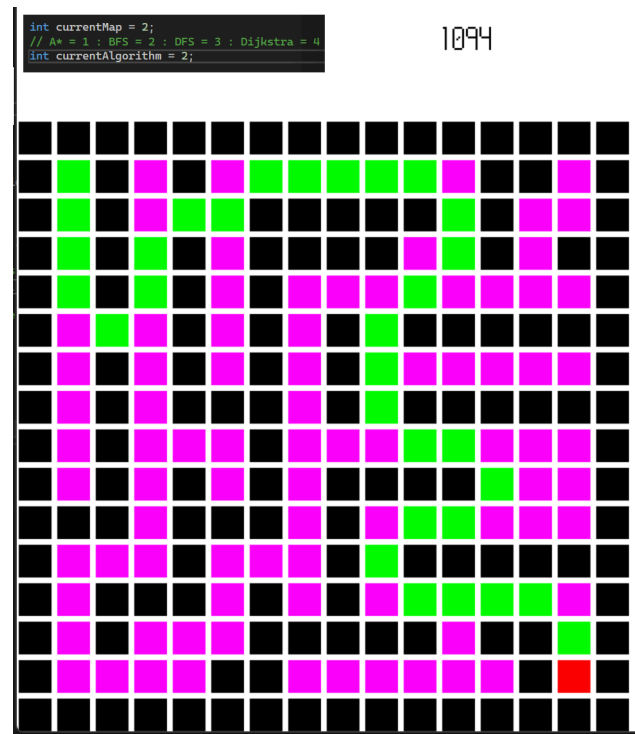
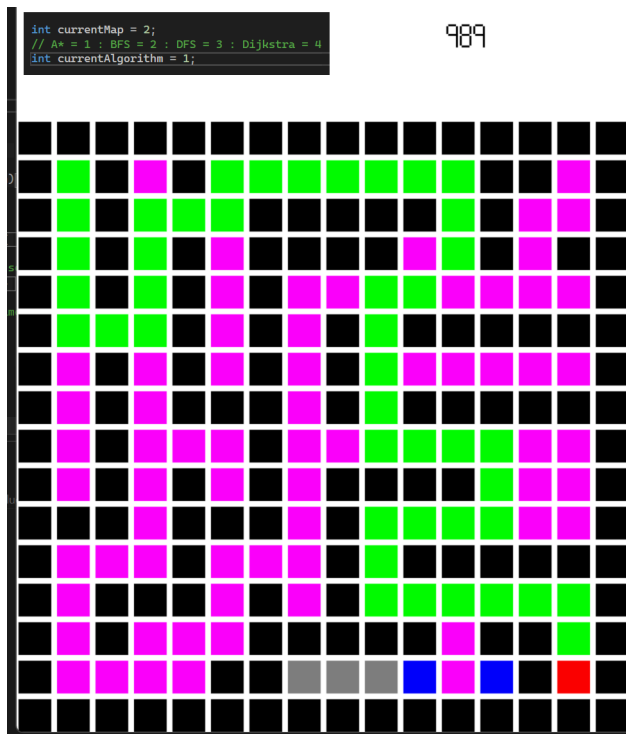
Top Left; A* algorithm

Top Right: Breadth First Search

Bottom Left; Depth First Search

Bottom Right; Dijkstra's Algorithm

Map 2 results



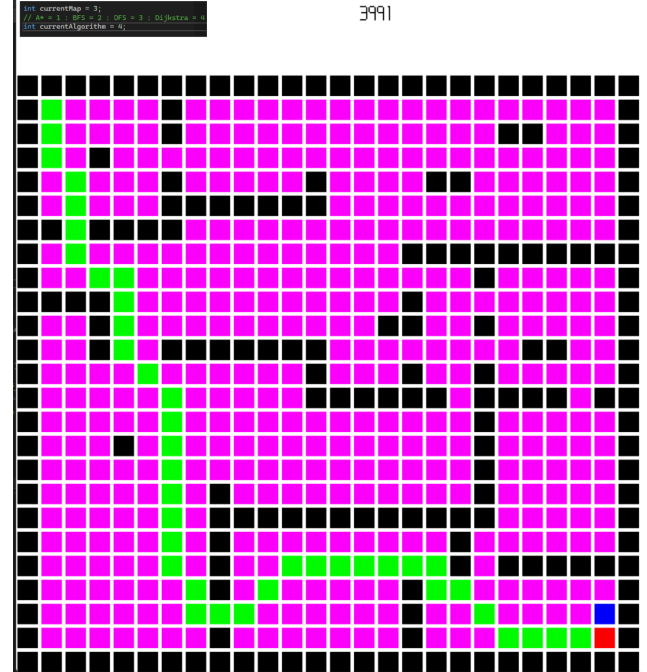
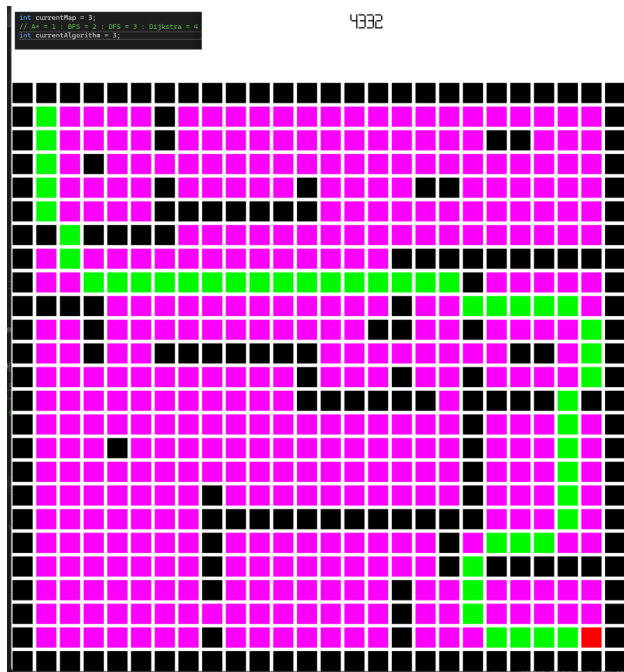
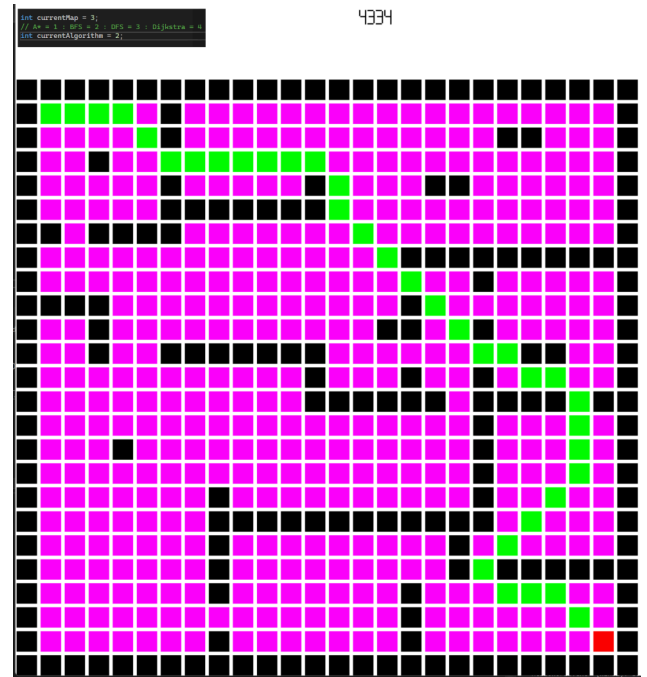
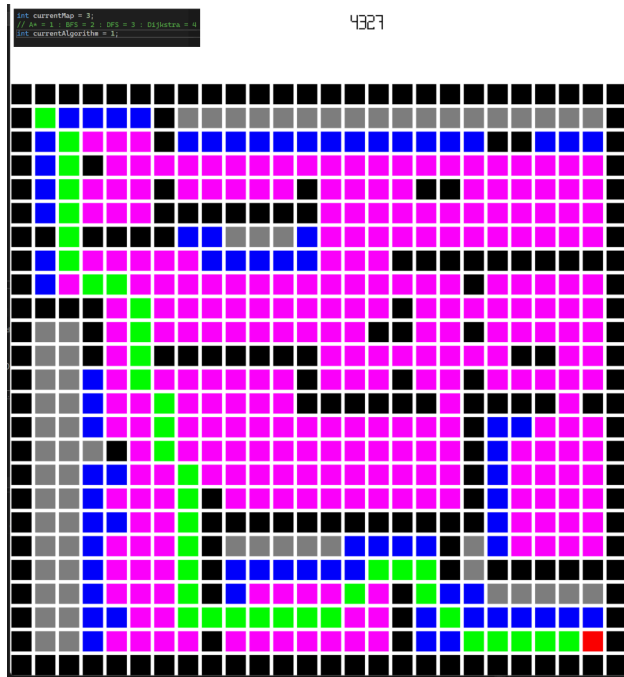
Top Left; A* algorithm

Top Right: Breadth First Search

Bottom Left; Depth First Search

Bottom Right; Dijkstra's Algorithm

Map 3 results



Top Left; A* algorithm

Top Right: Breadth First Search

Bottom Left; Depth First Search

Bottom Right; Dijkstra's Algorithm